

# LAD Prototype Development Cycle — From Backend Foundation to Frontend Experience

---

This account describes how I developed the LAD learning and academic management prototype through successive working versions. It is based on the 91 commits reachable from the reviewed repository revision, covering 5 April to 5 September 2026, together with the application code and development notes. The reviewed revision is [cc15bf5](#).

I began by establishing the backend data structure and user-management foundation, then introduced the frontend and connected it to the API. Backend and frontend development subsequently progressed together: each new academic workflow required supporting data, processing rules, access controls, and screens. The history therefore supports an iterative development cycle, with repeated implementation, integration, testing, demonstration, and refinement.

**1. Defining the prototype around academic workflows.** The system was organised around the work of administrators, trainers, and learners. Administrators needed to manage institutions, academic structures, users, and reports. Trainers needed to manage assigned learners, assessments, marks, attendance, and feedback. Learners needed access to their subjects, results, attendance, practical work, and progress information.

These needs became the functional structure of the prototype. I used shared academic records so that an action in one area could support another—for example, a trainer's recorded assessment could contribute to a learner's results and an authorised administrator's report. This description of the initial scope is reconstructed from the implemented modules; the reviewed material does not establish a separate requirements-sign-off date.

**2. Building the backend and database foundation.** The first substantive backend commit, [e9ddf64](#) on 5 April 2026, introduced the Flask application, user and academic models, authentication routes, database migrations, and initial tests. The early models included institutions, departments, courses, students, trainers, users, and roles.

I used Python and Flask to expose application functions through an API, with PostgreSQL for persistent records and SQLAlchemy to express their relationships. Flask-Migrate and Alembic migrations recorded changes to the database structure as the prototype expanded. Later additions included subjects, modules, enrolments, assessments, scores, attendance sessions, practical assessment records, and learner feedback.

The backend grew into three main responsibilities: models define stored information; routes receive requests and return responses; services contain shared processing and reporting logic. This structure allowed multiple screens to use the same academic records and calculations.

**3. Establishing authentication and permissions.** User access was part of the early implementation. The 6 April commit [aa7f4e5](#) expanded authentication, role management, user administration, and permission-aware screens. The application uses password hashing and signed, time-limited authentication tokens. The frontend sends the token with API requests, and the backend checks the requesting user and applicable permissions.

As reporting became more detailed, access control also developed beyond permission to open a page. Reporting queries were refined to restrict the records included according to the user's teaching assignments or oversight responsibilities. This distinction matters because someone may be allowed to view a report while being entitled to see only their own learners, department, or institution.

**4. Creating the frontend and connecting it to the API.** Frontend work began on 5 April in commit [47eb7e2](#), after the initial backend foundation. I used React and TypeScript, with Vite for development and builds. The first interface included login and registration screens, a dashboard layout, navigation, summary cards, tables, and charts. Trainer-management and trainer-dashboard pages followed on 6 April in [73fe071](#).

I connected the screens to the backend through API request helpers and authentication state. TypeScript definitions added on 11 April in [c0f155a](#) helped the interface work with the backend's data structures. Shared components for forms, tables, modals, navigation, and charts provided a consistent interface as more modules were added.

The current interface uses protected routes and permission-aware navigation to direct users to relevant functions. Backend checks also apply to API requests. A typical interaction follows this sequence:

1. A user opens a screen or submits a form in the React dashboard.
2. The frontend sends an authenticated request to the Flask API.
3. The backend checks access, validates the request, and reads or updates PostgreSQL records.
4. Shared services calculate any required summaries or report values.
5. The API returns data or an error response, and the frontend updates the relevant table, card, chart, or form.

**5. Expanding into complete academic workflows.** During May and June, I extended the initial management screens into connected workflows. The 11 May commit [9b541ea](#) improved table controls, common styling, dashboard API integration, and summary panels. On 13 May, [6510ca4](#) introduced an attendance-session structure with backend models and services, trainer session screens, and learner check-in screens, including QR and location-related functionality.

By 14 June, commit [1ae3f9c](#) added or expanded online examination support, student reports, score evidence, and report-permission services. These changes connected assessment records with supporting documents and reporting functions. Further iterations expanded practical assessments, competency tracking, portfolio evidence, notifications, and feedback.

Bulk entry also became an important workflow. On 2 August, [f50fcfe](#) added an Excel marks-upload template, while [057f895](#) enabled trainers to create assessments for bulk marks. Together, these changes supported entering class results through an assessment-linked upload process.

**6. Developing analytics and improving the meaning of reports.** As academic records accumulated, I added dashboards and reports to present performance, attendance, competency mastery, enrolment, and learners who might need support. The implementation includes shared analytics services and chart components so that recorded activity can be presented as useful summaries. Recommendations in the reviewed code are rule-based suggestions derived from academic signals.

Later work focused on whether the figures were both correctly calculated and correctly scoped. On 28 August, [44eed1a](#) refined cross-cohort reporting access and mastery calculations. On 5 September, [cc15bf5](#) standardised more of the percentage, grading, and pass-status logic across entry and reporting paths.

One concrete example appears in `backend/app/routes/admin_reports_v2.py` : earlier report logic could compare a raw mark of 45 against a pass threshold of 50 even when the assessment was marked out of 50. The revised calculation converts 45/50 to 90%, allowing different assessment totals to be compared on a common percentage scale. This illustrates how development moved from displaying data to checking that the displayed figures had the intended academic meaning.

**7. Testing with linked demonstration data.** Automated tests were present in the initial backend work and expanded alongside the application. The current test files cover areas including authentication, user and role creation, permissions, record scoping, bulk imports, marks calculations, online exams, practical assessments, notifications, and syllabus-coverage calculations.

I also added scripts to create linked demonstration records. Early seeding scripts were committed on 6 May in `b807283` . On 1 September, `afdb118` added a trainer-showcase script covering a trainer account, subjects, learners, marks, attendance, practical assessments, competencies, portfolio records, reports, and feedback. These linked scenarios provided data with which to demonstrate workflows across several screens.

Demonstration records are synthetic and do not establish real learner outcomes or measured improvements. The repository also documents backend test, frontend type-check, and frontend build commands. Test presence and these documented commands establish the available validation process; this development-history review did not execute the application checks or independently confirm historical pass results.

**8. Refining the prototype through stakeholder review.** The existing development journey records a stakeholder review dated 13 August 2026. Subsequent commits show how review feedback became changes to both the backend and the interface.

On 30 August, `fa84c59` removed redundant navigation and repeated guidance, clarified labels, refined report scoping, and introduced learner validation of syllabus coverage. Trainers could report topics as taught, learners could confirm whether those topics had been covered, and staff could compare the two perspectives. The calculation treats missing learner responses separately from disagreement, avoiding an automatic negative judgement where no feedback has been submitted.

September refinements continued this cycle. The 1 September update improved learner performance screens and demonstration coverage. On 3 September, `bd8267c` changed recommendations into language a learner could act on—for example, revision guidance instead of instructions to re-teach a topic—and scoped assessment-report filters to a trainer's teaching responsibilities. It also adjusted demonstration data to the active academic term. These changes addressed the clarity of the experience as well as the underlying data.

**Recorded development milestones.** These dates are Git author dates. Phase boundaries are descriptive groupings of the work, rather than evidence of formally scheduled sprints.

| Period       | Recorded development                                                                          | Representative commits                                             |
|--------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| 5 April 2026 | Backend models, authentication routes, migrations, initial tests, and first frontend scaffold | <code>e9ddf64</code> , <code>47eb7e2</code>                        |
| 6–11 April   | User and role management, trainer screens, API integration, and typed data structures         | <code>aa7f4e5</code> , <code>73fe071</code> , <code>c0f155a</code> |
| May          | Linked demonstration data, dashboard integration, table improvements, and attendance sessions | <code>b807283</code> , <code>9b541ea</code> , <code>6510ca4</code> |

| Period            | Recorded development                                                                                        | Representative commits                                                                                |
|-------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| June              | Online exams, score evidence, student reporting, and finer report permissions                               | <a href="#">1ae3f9c</a>                                                                               |
| July–early August | Continued workflow fixes and assessment-linked bulk marks entry                                             | <a href="#">a45a70a</a> , <a href="#">f316a18</a> , <a href="#">f50fcfe</a> , <a href="#">057f895</a> |
| 28–30 August      | Reporting and mastery corrections, user guidance, simpler navigation, and learner coverage validation       | <a href="#">44eed1a</a> , <a href="#">fa84c59</a>                                                     |
| 1–5 September     | Fuller demonstrations, learner-facing refinements, scoped filters, and more consistent grading calculations | <a href="#">afdb118</a> , <a href="#">bd8267c</a> , <a href="#">cc15bf5</a>                           |

**9. Preserving versions and preparing for evaluation.** Git records the incremental changes, and the repository contains two annotated stage tags: [v1.0-prototype-baseline](#) and [v1.1-review-refinements](#). User and permissions guides document how the resulting application is intended to be used. Setup instructions, database migrations, and demonstration scripts support recreating the prototype for review.

The tag dates require a precise distinction: both tags were created on 30 August. The baseline tag points to [44eed1a](#), dated 28 August, and the refinement tag points to [fa84c59](#), dated 30 August. Although the earlier development notes associate the baseline with the 13 August review, the tagged baseline already includes the 28 August changes. It should therefore be described as the preserved baseline before the 30 August refinement commit, rather than an independently verified snapshot of the exact code shown on 13 August.

The reviewed history establishes an implemented prototype with successive integration and review changes. It does not by itself establish formal customer acceptance, completion of a production rollout, or a completed evaluation period. The existing development notes identify a system freeze for evaluation as a planned step.

**Repository references.** The narrative can be checked against the following materials:

- Project overview and setup ( [../README.md](#) )
- Existing stakeholder-review development journey ( [DEVELOPMENT-JOURNEY.md](#) )
- Backend application setup ( [../backend/app/\\_\\_init\\_\\_.py](#) ), models ( [../backend/app/models/](#) ), and migrations ( [../backend/migrations/versions/](#) )
- Authentication ( [../backend/app/routes/auth.py](#) ) and shared reporting calculations and scoping ( [../backend/app/services/scoping.py](#) )
- Administrative reporting ( [../backend/app/routes/admin\\_reports\\_v2.py](#) ) and learning analytics ( [../backend/app/services/learning\\_analytics.py](#) )
- Frontend routes ( [../dashboard/src/App.tsx](#) ), API client ( [../dashboard/src/api/client.ts](#) ), and backend data types ( [../dashboard/src/types/backend.ts](#) )
- Backend tests ( [../backend/tests/](#) ) and trainer demonstration script ( [../backend/scripts/seed\\_trainer\\_showcase.py](#) )
- User guide ( [LAD-User-Guide.pdf](#) ) and permissions guide ( [LAD-Permissions-Guide.pdf](#) )

Individual milestones can be inspected with `git show <commit>`. The preserved refinement round can be compared with `git diff v1.0-prototype-baseline..v1.1-review-refinements`.